키프레임 고정 객체 지도: 비동기 SLAM에서 늦게 도착하는 제로샷 6D 자세 추정 결과의 등록

Keyframe-Anchored Object Maps: Registering Late Zero-Shot 6D Pose Estimates in Asynchronous SLAM

[저자·소속은 투고처 규정에 따라 기입 (익명 심사이면 별도 표지에)]

요약

CAD 모델만으로 물체를 인식하는 제로샷 6D 자세 추정기는 한 장에 수백 ms에서 수 초가 걸린다. 이를 SLAM과 다른 연산 장치에서 실행하면 SLAM은 실시간을 유지하지만, 인식 결과는 다른 카메라와 다른 시계에서 촬영 후 0.7~6.6초 늦게 도착하고 그 사이 루프 클로저로 지도가 보정될 수 있다. 본 논문은 이렇게 늦게 도착하는 결과로 지속적인 키프레임 고정 객체 지도를 구축하는 두 장치 시스템을 제안한다. 핵심인 촬영 시각 키프레임 고정은 SLAM 장치가 자세마다 추적에 쓴 기준 키프레임을 함께 보내고 키프레임 목록을 다시 보내며, 인식 장치는 각 물체를 영상 촬영 시각의 기준 키프레임에 대한 상대 자세로 저장한다. 여기에 메타데이터 촬영 시각과 시간 오프셋·외부 파라미터 동시 추정, 텍스트 프롬프트와 순차 관문으로 검증하는 SAM-6D, 존재확률 기반 상태 관리, 인식 지연 보상 표시를 결합한다. 실시간 실행 기록 61회를 재생한 실험에서 같은 물체 관측의 흩어짐 중앙값은 9.67 cm에서 1.51 cm로, 결과를 기다리는 동안 지도가 보정된 경우 10 cm를 넘게 어긋난 비율은 30.8%에서 1.5%로 줄었다. 실내 주행에서 화면을 벗어났다 다시 보인 물체 74건 중 72건이 같은 물체로 재연결되었다(지도 없이 34건).

주제어

비동기 센서 융합, 루프 클로저, 객체 수준 지도, SLAM, 제로샷 6D 객체 자세 추정

Abstract

Zero-shot 6D pose estimators recognize objects from CAD models alone but take up to seconds per frame. Running them on a second computer keeps SLAM real-time, yet results arrive 0.7–6.6 s late, from another camera and clock, after loop closure may have corrected the map. This study presents a two-computer system that turns such late results into a persistent object map. Its core, capture-time keyframe anchoring, streams the reference keyframe of every SLAM pose with re-sent keyframe lists and stores each object relative to the keyframe current at image capture. Metadata timestamps with joint offset–extrinsic calibration, a text-prompted and gate-verified SAM-6D, an existence-probability state manager, and a latency-compensated display complete the system. On 61 replayed live runs, anchoring reduces the median deviation of repeated observations from 9.67 to 1.51 cm and, when the map is corrected in flight, the share above 10 cm from 30.8% to 1.5%. Indoors, 72 of 74 reappearing objects are re-associated (34 without the map).

Keywords

asynchronous sensor fusion, loop closure, object-level mapping, simultaneous localization and mapping, zero-shot 6D object pose estimation

1 | 서론

서비스·물류 로봇, 디지털 트윈, XR 응용에는 공간 안의 물체마다 지도 좌표계의 3차원 위치와 방향을 담고, 플랫폼이 움직이는 동안 계속 갱신되는 *객체 수준 지도*가 필요하다. 이를 위한 기술은 두 갈래로 발전해 왔다. ORB-SLAM3 [1], HDL-Graph-SLAM [2] 같은 영상·LiDAR SLAM은 센서 자세를 실시간으로 추정하고 루프 클로저와 그래프 최적화로 누적 오차를 없앤다. SAM-6D [3], FoundPose [4], GigaPose [5] 같은 제로샷 6D 객체 자세 추정기는 학습에 쓰지 않은 물체도 CAD 모델만으로 자세를 추정한다.

두 기술을 합치는 일은 변환 두 개를 곱하는 것으로 끝나지 않는다. 제로샷 추정기는 느리다. SAM-6D는 화면 전체를 수백 개의 후보로 분할하여 본 실험 환경에서 한 장에 약 0.9초가 걸린다. 추정기가 SLAM과 같은 연산 장치를 쓰면 SLAM이 처리 시간을 잃는다. 본 시스템에서 같은 장치에 무거운 처리를 함께 실행하자 SLAM 자세의 전달 지연이 0.04초에서 2.5초로 늘었다. SLAM이 느려지면 추적이 불안정해지고 큰 지도 보정이 잦아져, 지도에 등록된 모든 물체가 함께 흔들린다. 인식과 SLAM을 함께 실행하는 Fusion++ [6]는 전체가 초당 4~8회로 동작한다.

인식을 다른 연산 장치로 옮기면 SLAM은 실시간을 유지하지만 다섯 가지 문제가 한꺼번에 생긴다. (i) 두 장치의 시계가 다르다. (ii) 두 카메라의 위치·방

향이 다르다. (iii) 인식 결과는 촬영 후 수백 ms에서 수 초 뒤에 도착하므로, *도착* 시각의 카메라 자세로 등록하면 그 사이 플랫폼이 움직인 만큼 물체가 어긋난다. (iv) 결과를 기다리는 동안 SLAM이 루프 클로저로 과거 키프레임 자세를 고치거나 키프레임을 정리(culling)한다. (v) 네트워크 부하에 따라서는 촬영 시각의 SLAM 자세가 인식 결과보다 *늦게* 도착한다.

이 가운데 (iv)가 가장 드러나지 않는 문제다. ROS tf [7]처럼 시각별 변환을 보관하는 방식으로 촬영 시각의 궤적을 보간하면 (iii)은 풀리지만 물체는 지도 좌표에 고정 저장되므로, 루프 클로저로 궤적이 옮겨지면 물체만 이전 위치에 남는다. 그림 1의 파랑 영역은 실제 사례이다. 키프레임을 최대 3 m 옮긴 루프 클로저 뒤에, 이 방식으로 저장한 결과 119건 중 68건이 최종 물체 위치에서 10 cm 넘게 떨어졌으나, 본 논문의 방식으로는 6건이었다. 증강현실에서 가상 객체를 키프레임에 붙이는 방법이 알려져 있고 ORB-SLAM3도 지도점을 기준 키프레임과 함께 옮기지만, 두 경우 모두 붙일 대상이 현재 프레임에서 바로 생긴다. 영상보다 수 초 늦게 계산된 결과를 *어느 시각의 어느 키프레임*에 붙일지는 다루어지지 않았다. 본 논문의 기여는 다음과 같다.

- **촬영 시각 키프레임 고정.** SLAM 장치는 자세마다 그 프레임의 추적에 쓴 기준 키프레임을 함께 보내고, 전체 키프레임 목록을 주기적으로, 지도 변경 직후, 그리고 0.5초·3초 뒤에 다시 보낸다. 인식 장치는 늦게 도착한 결과를 촬영 시각의 기준 키프레임에 대한 상대 자세로 저장하고, 해당 자세가 아직 없으면 기다렸다가 등록하며, 정리된 키프레임의 물체는 남은 키프레임으로 옮긴다.
- **장치 간 시간·외부 파라미터 정렬.** 카메라가 프레임마다 주는 메타데이터 시각, 왕복 시간으로 구한 시계 차이, 그리고 잔여 시간 오프셋을 카메라 간 외부 파라미터와 함께 추정하는 세 단계 보정을 사용한다.
- **실시간으로 동작하고 검증 가능한 제로샷 인식기.** 텍스트 프롬프트 후보 생성, 순차 관문, 자세 검증으로 SAM-6D를 바꾸어 물체가 없으면 '없음'으로 답할 수 있게 한다.
- **키프레임 고정 객체 지도와 인식 지연 보상 표시.** YCB-Video [8], YCB-M [9], 그리고 하드웨어 동기화한 RGB-D 카메라 두 대와 LiDAR로 녹화한 실내 주행에서 평가하고 한계를 함께 제시한다.

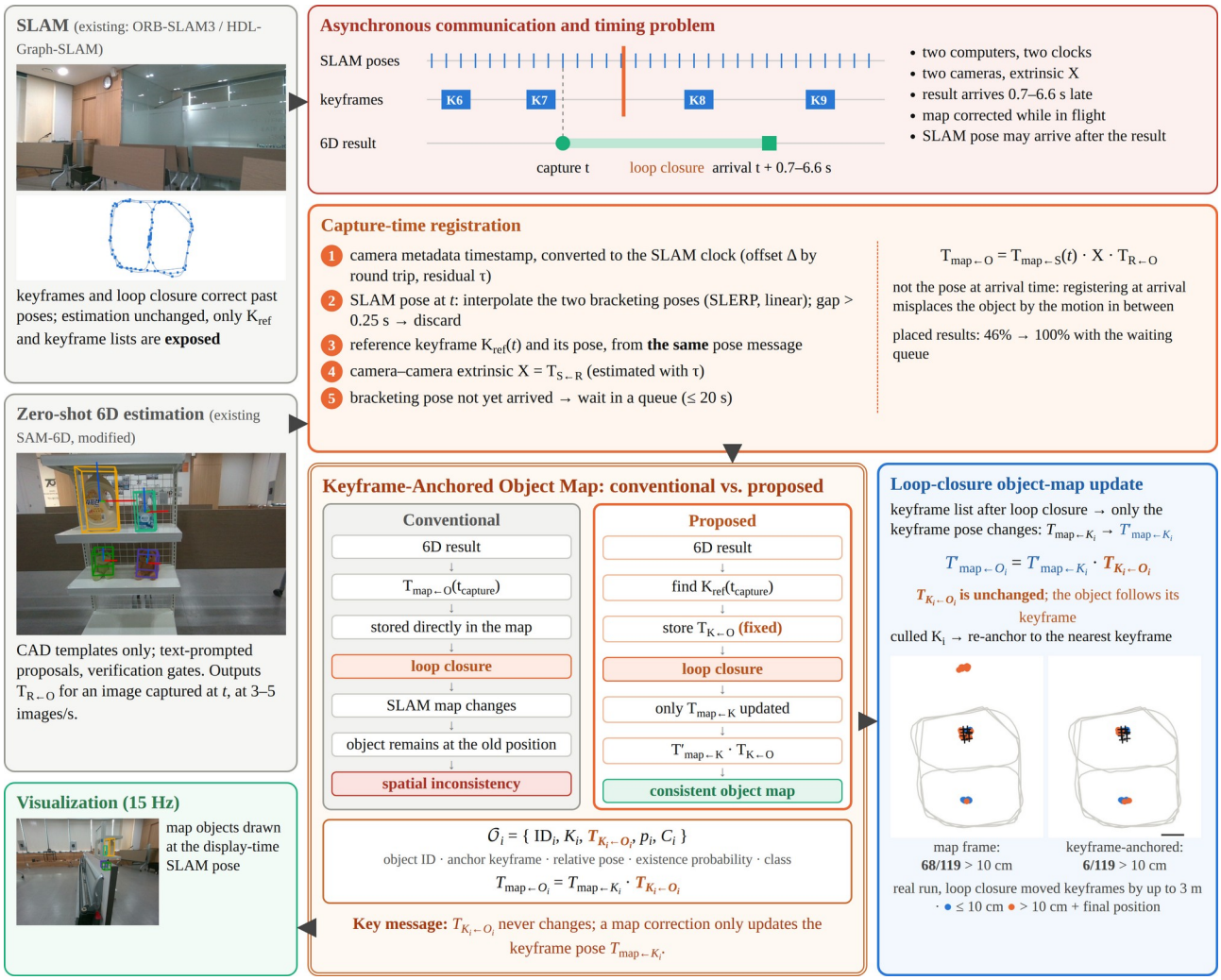


그림 1 전체 구성도. 각 영역의 면적은 역할의 비중에 맞췄다. 기존 SLAM과 제로샷 6D 추정(회색)이 두 연산 장치에서 동작하므로, 결과는 다른 시계와 카메라에서 늦게 도착하고 그 사이 지도가 보정될 수 있다(빨강). 각 결과는 촬영 시각으로 등록하고(주황) 그 시각의 기준 키프레임에 대한 상대 자세로 저장한다(이중 테두리). 지도 좌표에 바로 저장하는 기존 방식과 달리, 루프 클로저 뒤 키프레임 보정을 따라 물체가 이동한다(파랑). 파랑 영역의 지도는 루프 클로저가 키프레임을 최대 3 m 옮긴 실제 주행이며, 모든 영상은 주행 녹화와 실행 기록에서 얻었다.

2 | 관련 연구

제로샷 6D 자세 추정. CNOS [10]와 SAM-6D [3]는 SAM [11] 또는 FastSAM [12]으로 영상을 분할하고, 후보마다 DINOv2 [13] 특징을 구해 CAD 모델로 렌더링한 템플릿과 비교한다. SAM-6D는 이어서 두 단계 점 대응으로 자세를 계산한다. FoundPose [4], GigaPose [5], FoundationPose [14]도 템플릿 또는 모델 기반의 비슷한 구조를 쓴다. 이들은 한 장의 영상에 대해 카메라 기준 자세를 내며 물체를 시간에 걸쳐 유지하지 않고, 처리 시간의 대부분은 후보 생성 단계에서 나온다. YOLO-World [15] 같은 개방 어휘 검출기는 텍스트 프롬프트로 수십 ms 안에 종류 수준의 상자를 내므로, 본 논문은 이를 개체 수준 검증 앞의 후보 생성기로 쓴다.

객체 SLAM과 객체 지도. SLAM++ [16], CubeSLAM [17], QuadricSLAM [18]은 미리 학습한 검출기나 종류별 형상을 이용해 물체를 랜드마크로 삼아 위치 추정을 개선한다. CosyPose [19]는 여러 시점의 객체 자세를 하나의 일관된 장면으로 모은다. Fusion++ [6], POCD [20], Khronos [21]는 존재·변화 모델을 둔 객체 수준 지도를 유지한다. 본 논문은 이들과 상호 보완적이다. 물체를 SLAM에 되먹이지 않고 SLAM의 추정은 그대로 둔 채, CAD로 정의한 물체가 SLAM의 보정을 따라가게 한다.

비동기 인식. MaskFusion [22]은 같은 연산 장치의 별도 GPU에서 인스턴스 분할을 비동기로 수행하며 카메라 하나와 시계 하나를 전제로 한다. 엣지 지원 모바일 AR [23]은 검출을 서버에 맡기고 돌아온 상자를 영상 추적으로 보정하지만 지도를 갖지 않는다. 변환 라이브러리 [7]는 요청한 시각의 자세를 보간하지만, 궤적이 최적화된 뒤 이미 저장한 결과를 다시 계산하지 않는다. 다른 장치에서 늦게 도착한 인식 결과를 촬영 시각의 SLAM 키프레임에 묶어, 결과를

기다리는 동안 일어난 보정까지 객체 지도에 반영하는 시스템은 저자들이 아는 범위에서 보고되지 않았다.

3 | 시스템 구성

그림 1은 전체 시스템이다. SLAM 장치(Device 1)는 자신의 센서로 SLAM을 수행하며, ORB-SLAM3에는 RGB-D 카메라를, HDL-Graph-SLAM에는 LiDAR를 쓴다. 인식 장치(Device 2)는 GPU와 인식용 RGB-D 카메라를 가진다. 두 장치는 유선 LAN으로 연결되고, 카트에 고정한 두 카메라는 하드웨어 신호로 동기화한다(그림 2). 표기는 다음과 같다. $T_{A\leftarrow B}$는 좌표계 B의 좌표를 A로 옮기는 변환이고, $X = T_{S\leftarrow R}$는 인식 카메라 R에서 SLAM 카메라 S로의 외부 파라미터이다. 시각 t에 촬영한 영상에 대해 추정기는 물체 O의 $T_{R\leftarrow O}$를 출력한다.

4 | 제안 방법

4.1 | 시간과 외부 파라미터 정렬



그림 2 데이터 수집 카트. 동기 케이블로 연결한 RGB-D 카메라 두 대(640 × 480, 30 Hz)와 16채널 LiDAR(10 Hz).

촬영 시각. 카메라가 프레임마다 메타데이터에 붙이는 시각을 쓴다(사용한 RGB-D 카메라에서는 카메라 하드웨어 시계를 연산 장치 시계로 환산한 값 또는 연산 장치 도착 시각). 실시간 동작에서는 프레임을 받는 즉시 읽고, 재생할 때는 녹화 파일에 저장된 같은 값을 읽는다. 녹화 프로그램의 연관 타임스탬프는 쓸 수 없었다. 프레임 누락(100~200 ms 간격) 때문에 실제 프레임 시각과 −2.1초에서 +3.8초까지 어긋났지만, 메타데이터 시각은 SLAM 카메라에서 1.4 ms, 인식 카메라에서 0.1 ms 이내로 일치하였다.

시계 차이. 인식 장치가 시각 t_0에 UDP 패킷을 보내면 SLAM 장치가 자기 시계 시각 t_r을 담아 돌려보내고, 인식 장치가 이를 t_1에 받는다. 시계 차이

$$\Delta = t_r - \left(t_0 + (t_1 - t_0) / 2 \right) \qquad (1)$$

는 여러 번 잰 값의 중앙값으로 정한다. 편도 지연이 같다는 가정의 오차는 왕복 시간의 절반 이하이며, 측정한 왕복 시간이 3.08 ms이므로 최대 1.54 ms로 컬러·거리 영상을 짝짓는 허용 범위 20 ms보다 충분히 작다. 모든 시각은 SLAM 장치의 시계로 환산한다.

잔여 오프셋과 외부 파라미터. 시각 기준과 전송 지연의 차이로 잔여 오프셋 τ가 남는다. 두 카메라가 각자 시각 주행거리를 구하는 보정 주행에서 τ와 X를 함께 추정하며, 평면 운동에서의 관측 가능성에 따라 세 단계로 나눈다. (1) 각 궤적에 맞춘 평면의 법선으로 중력 방향을 구한다(평면 이탈 RMS 0.60 cm, 0.20 cm). (2) 0.5~4초 구간의 상대 운동 쌍으로 $AX = XB$ 제약을 만들고, τ를 탐색하면서(예: ±60 ms 범위, 2 ms 간격) Huber 가중 최적화로 수평 위치, 회전각, τ를 구한다(14 592쌍, 잔차 중앙값 0.41 cm). (3) 평면 운동으로는 관측되지 않는 높이 차는 각 거리 영상의 바닥면으로 잰다(1.107 m, 0.998 m). τ는 보정 주행 세 번에서 0~+4 ms, 다른 녹화에서 −48 ms, 녹화 장치 시계가 맞지 않았던 녹화에서 0.196초였고, 모두 같은 탐색으로 찾았다. 같은 방법으로 LiDAR–카메라 외부 파라미터도 구한다(3122쌍, 잔차 1.18 cm, 0.11°).

4.2 | SLAM 장치: 실시간 추적과 키프레임 전송

실시간 처리. ORB-SLAM3의 특징점을 프레임당 2000개로 늘리면 평균 추적 시간이 24.7 ms가 되고 7142프레임 중 170프레임이 카메라 주기 33.3 ms를 넘었다. 분석 결과 병목은 특징점 추출이나 최적화 반복이 아니라 지도점 수에 비례하는 부대 비용이었다. 계산 결과를 바꾸지 않는 일곱 가지 수정(꺼진 화면 표시용 프레임 복사 제거, 지도점 관측의 제자리 집계, 할당 없는 영역 검색, 비트 계수 명령을 쓴 기술자 거리 계산, 지도점 조회 잠금을 네 번에서 한 번으로 통합, 분배 트리 메모리 예약 축소, 피라미드 단계별 병렬 추출)으로 평균은 15.4 ms, 주기를 넘긴 프레임은 0개가 되었고, 8자 주행의 귀환 오차는 26.6 cm에서 26.5 cm로 변하지 않았다(그림 3). 처리가 밀리면 가장 최근 프레임을 처리하고 건너뛴 프레임 수를 알린다.

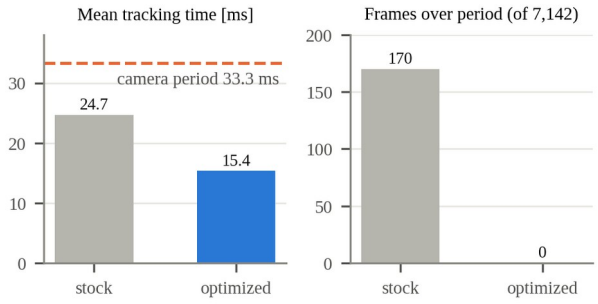


그림 3 특징점 2000개 ORB-SLAM3의 결과 불변 최적화 전후 처리 시간.

자세 메시지. SLAM 장치는 프레임마다 촬영 시각, 추적 상태, $T_{map \leftarrow S}$(추적에 실패했거나 사전 지도 모드에서 지도에 고정되지 않았으면 '자세 없음'), 그 프레임의 추적에 쓴 기준 키프레임의 번호와 자세 $T_{map \leftarrow K}$, 지도 번호, 지도 변경 여부, 건너뛴 프레임 수를 보낸다. 자세와 기준 키프레임은 추적 직후 같은 추적 스레드에서 읽으므로 같은 순간의 값이다. ORB-SLAM3는 이 정보를 외부로 제공하지 않으므로 읽기 전용 조회 기능을 추가하였다. 자세 전달 지연의 중앙값은 18.1 ms이다.

키프레임 목록. ORB-SLAM3의 루프 클로저 뒤에는 별도 스레드의 전역 번들 조정이 수 초 뒤에 끝나므로, 루프 직후 한 번만 보내면 최종 보정이 빠진다. 그래서 SLAM 장치는 정리되지 않은 모든 키프레임의 목록(번호와 3 × 4 자세, 키프레임 100개에 약 14 kB)을 1초마다, 지도 변경이 표시된 프레임 직후, 그리고 0.5초와 3초 뒤에 다시 보내며, 겹치는 전송은 하나로 합친다. 목록에서 빠진 키프레임은 정리된 것으로 보므로 별도의 정리 통지가 필요 없다. HDL-Graph-SLAM에는 그래프 최적화된 키프레임 자세 전체를 발행하는 기능을 추가하여 LiDAR SLAM에도 같은 고정 방식을 적용하였다.

4.3 | 텍스트 프롬프트와 관문 검증을 쓴 6D 자세 추정

그림 4는 인식기를 SAM-6D와 단계별로 비교한 것이다. 템플릿은 42개 시점에서 렌더링한다. SAM-6D의 DINOv2 요약 벡터와 자세 추정용 특징에 더해, DINOv2 중간층 지역 특징, 렌더링 색과 카메라 색의 차이를 템플릿 쪽에서 보정한 16 × 8 색조·채도 분포, 물체별 텍스트 임베딩을 미리 계산한다. 새 물체는 CAD 모델과 한 번의 사전 계산만으로 추가된다.

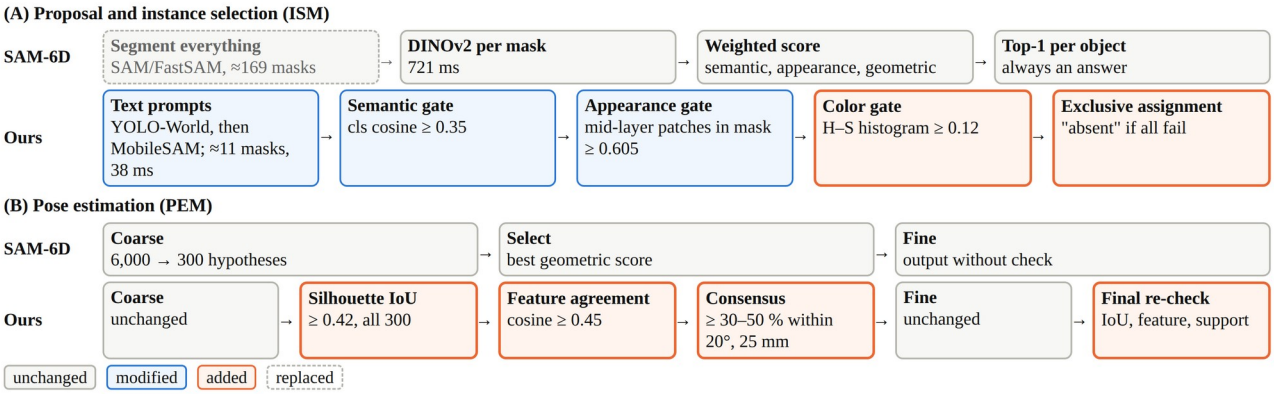


그림 4 SAM-6D와 제안 인식기의 비교. (A) 화면 전체 분할 후보와 가중 합 점수를 텍스트 프롬프트 후보와 순차 관문으로 바꾼다. (B) 정밀화 전후에 자세 검증을 넣는다.

후보 생성. 모든 물체의 프롬프트(예: "milk carton", "yellow can")를 YOLO-World [15]에 한 번에 넣고, 낮은 신뢰도 기준(0.02) 이상의 상자를 남기며 여러 프롬프트에 걸린 상자는 모두 남긴다. MobileSAM [24]이 상자 안 물체의 마스크를 만든다. 프레임당 후보는 169개에서 11개로, 후보 특징 추출 시간은 721 ms에서 38 ms로 줄었다. 대상 물체가 없는 193장에서 오검출이 생긴 장은 120장에서 2장으로 줄었다(대상이 있는 118장의 재현율 0.949 → 0.915).

순차 관문. 가중 합 뒤 물체마다 1등 후보를 고르는 대신, 각 후보는 다음 관문을 차례로 통과해야 한다. (1) 의미 관문: 가장 닮은 템플릿 5개의 요약 벡터 코사인 평균 ≥ 0.35. (2) 외형 관문: 마스크 안 중간층(2번·9번 블록) 지역 특징의 최대 일치 코사인 평균 ≥ 0.605. (3) 색 분포 관문: 밝기를 뺀 색조·채도 분포의 Bhattacharyya 유사도 ≥ 0.12. (4) 배타 판정: 한 상자는 한 물체에만 배정하며, 템플릿끼리 닮은 물체 사이에서는 색 분포 점수로 정한다. 남은 후보가 없는 물체는 '없음'으로 답한다.

자세 검증. SAM-6D 자세 추정 모델의 coarse(가설 6000개 → 300개)와 fine 단계는 그대로 둔다. 정밀화 전에 300개 가설 모두에 대해 투영한 CAD 점의 실루엣과 마스크의 IoU(≥ 0.42), 관측 점과 대응 CAD 점의 특징 코사인 평균(≥ 0.45), 수렴(통과 가설의 30~50% 이상이 한 자세의 20°·25 mm 안에 모임)을 확인한다. 정밀화한 자세도 다시 확인한다.

4.4 | 키프레임 고정 객체 지도

촬영 시각 등록. 시각 t에 촬영한 영상의 결과는

$$T_{map\leftarrow O} = T_{map\leftarrow S}(t) \cdot X \cdot T_{R\leftarrow O} \quad (2)$$

로 지도 좌표에 옮긴다. $T_{map\leftarrow S}(t)$는 t를 감싸는 두 SLAM 자세를 보간한 값이며(회전은 구면 선형 보간, 위치는 선형 보간), 두 자세의 간격이 0.25초를 넘으면 그 검출은 쓰지 않는다. 물체는 t에 가까운 쪽 자세의 기준 키프레임 K에 대한 상대 자세로 저장하며, 이때 *그 자세 메시지와 함께 온* 키프레임 자세를 쓴다.

$$T_{K\leftarrow O} = T_{map\leftarrow K}(t)^{-1} \cdot T_{map\leftarrow O} \quad (3)$$

사용할 때는 키프레임 목록으로 받은 최신 키프레임 자세 T'를 곱한다.

$$T'_{map\leftarrow O} = T'_{map\leftarrow K} \cdot T_{K\leftarrow O} \quad (4)$$

즉 각 물체는 $O_i = \{ID_i, K_i, T_{K_\{i}\leftarrow O_i\}, p_i, C_i\}$로 저장된다. ID_i는 물체 번호, K_i는 고정 키프레임, $T_{K_\{i}\leftarrow O_i\}$는 상대 자세, p_i는 존재확률, C_i는 물체 종류이다. 지도가 보정되면 $T_{map\leftarrow K_\{i}\}$만 바뀌고 $T_{K_\{i}\leftarrow O_i\}$는 바뀌지 않는다.

$T_{map\leftarrow S}(t)$와 $T_{map\leftarrow K}(t)$는 같은 추적 상태에서 나오므로, 식 (3)은 t 이후 K에 가해진 어떤 보정에도 변하지 않으며 결과가 도착하기 전에 일어난 보정도 마찬가지다(그림 1). 반면 *도착* 시각의 키프레임에 붙이면 보정 전 카메라 자세와 보정 후 키프레임 자세를 섞게 된다. 감싸는 SLAM 자세가 아직 도착하지 않았으면 결과를 대기 목록에 최대 20초 두었다가 등록한다. K가 정리되면 물체를 가장 가까운 남은 키프레임으로 옮긴다. 키프레임 자세가 바뀔 때마다 기록된 관측 전체로 지도를 다시 만들 수 있다(308프레임에 75 ms).

연결과 갱신. 새 검출은 같은 종류이면서 0.15 m 안에 있는 기존 물체와 연결하며(확정 전 후보는 3배 범위), 한 프레임 안에서는 가까운 쌍부터 1:1로 짝짓는다. 제로샷 추정기는 상자형 물체를 180° 뒤집어 답하는 일이 잦아 한 물체가 둘로 쪼개지므로 회전은 연결 조건에서 뺀다. 위치는 관측의 누적 평균으로, 방향은 30° 안에 모이는 답끼리 표를 모은 최다 득표로 정하며, 회전 대칭 물체는 기준 방향에 가장 가까운 대칭 방향으로 정렬한 뒤 누적한다.

존재확률. 물체마다 존재확률 p와 상태를 둔다. 후보($p = 0.7$에서 시작, 표시 안 함), 확정(관측 2회 이상, $p > 0.6$), 놓침($p < 0.3$), 장기 기억($p < 0.05$이면서 관측 5회 이상), 삭제의 다섯 가지이다. 물체가 시야 안에 투영되고 *동시에* 인식기의 검출 가능 거리(예: 작은 물체 1.5~2.2 m) 안에 있을 때만 미검출로 세며, 이때 검출 확률 0.08~0.5와 오검출 비율 0.01~0.1로 베이즈 갱신한다. 거리 영상에서 물체 자리 뒤가 15 cm 이상 비어 보이는 경우가 반경 15 cm 창의 95% 이상에서 3프레임 연속 확인되면 치워진 물체로 보고 삭제한다. 놓침·장기 기억 상태의 물체도 연결 후보로 남기므로 다시 보이면 같은 번호를 되찾는다. 같은 종류 물체가 둘 이상이고 한쪽의 관측 수가 다른 쪽의 5배 이상이면 적은 쪽을 중복으로 보고 숨긴다. 이는 점수 기준으로는 거를 수 없는 높은 점수(0.96~1.00)의 오인식을 없앤다.

4.5 | 인식 지연 보상 표시

인식 장치는 인식 카메라 영상 최근 120장을 촬영 시각과 함께 보관한다. 초당 15회, 가장 최근 영상의 시각에서 표시 지연을 뺀 값을 표시 시각으로 정하고, 그 시각의 영상을 버퍼에서 골라 같은 시각의 SLAM 자세를 보간한 뒤 지도의 모든 물체를 그 영상에 투영한다. 따라서 영상과 상자는 서로 어긋나지 않고 화면 전체가 표시 지연만큼 늦을 뿐이다. 표시 지연은 최근 SLAM 자세 도착 지연의 상위 90% 값에 0.15초를 더한 값으로 하되 0.3~3초로 제한하며, 늘릴 때는 바로 늘리고 줄일 때는 한 번에 2%씩 줄인다. 표시 지연은 화면에만 적용되고, 로봇이 쓰는 지도는 최신 결과와 최신 키프레임 자세로 갱신된다.

5 | 실험 환경

장치. 카트(그림 2)에는 RGB-D 카메라 두 대와 16채널 LiDAR를 장착하였다. 녹화한 세션을 유선 LAN으로 연결한 두 연산 장치에서 각자 자기 센서 기록을 공통 시작 시각부터 1배속으로 재생하며 실시간으로 처리하였다. SLAM 장치는 Apple M4 Pro 노트북으로 ORB-SLAM3를 네이티브로 실행하고, 인식 장치는 Intel Core i9-12900HK와 NVIDIA GeForce RTX 3080 Ti Laptop GPU를 쓴 노트북이다 [실행별 확인 필요]. 한 장씩 비교한 실험(표 1)은 NVIDIA GeForce RTX 5090에서 수행하였다 [확인 필요]. 모든 실행에서 SLAM은 사전 지도 없이 실행 중에 지도를 처음부터 만들었다.

데이터. *YCB-Video* [8]: 평가용 영상 12개. 한 장 평가에는 BOP [25] 평가 목록 900장을, 실시간 평가에는 20 738프레임 전체를 30 fps로 재생하였다. *YCB-M* [9]: 로봇 팔 카메라로 정지 장면 32개를 찍은 7252장. 한 장 평가에는 8장마다 1장(925장)을 쓰고, 촬영 시각이 없어 재생 속도는 프레임 간 카메라 이동량(약 13 mm)으로 정하였다. *실내 주행*: 물체 8종(우유팩, 초코과자, 탈취제, 머그컵, 사프란, 식혜캔, 곰인형, 공룡인형)을 실내 공간에 놓고 녹화한 어두운 실내 8자 주행(284초), 91초 실내 주행(1883프레임, 지도 보정 96회), 밝은 실내 홀 8자 주행(151초), 큰 8자 주행(315초). 실내 주행에는 정답 자세가 없으므로 같은 물체 관측의 일관성과, 인식 카메라 영상으로

따로 최적화한 SLAM 궤적에서 만든 의사 정답을 쓴다.

지표. ADD-S가 물체 지름의 10% 미만이면 맞는 자세로 본다. *찾은 비율*은 보이는 정답 물체 중 맞는 답이 있는 비율, *맞는 답 비율*은 낸 답 중 맞는 답의 비율이며, BOP AR은 공식 도구로 계산한다. *화면 일치율*은 매 프레임 보이는 정답 물체 중 화면 상자가 맞는 비율이다. *흩어짐*은 최종 지도에 등록된 관측이 그 물체의 중앙 위치에서 떨어진 거리이다.

6 | 실험 결과

6.1 | 인식

표 1은 SAM-6D(공식 코드, FastSAM 후보)와 텍스트 프롬프트를 쓴 제안 인식기, 그리고 후보 재현율의 영향을 분리하기 위해 텍스트 검출기 대신 정답 상자를 넣은 제안 인식기를 비교한다. 제안 인식기는 세 데이터 모두에서 낸 답의 94~100%가 맞고, 틀린 답은 원본의 1/7 이하이며 3.7~5.6배 빠르다. 실내 주행에서는 BOP AR도 원본보다 높지만(37.9 대 33.3), YCB에서는 텍스트 검출기가 물체를 놓쳐 찾은 비율과 AR이 원본보다 낮다.

표 1 SLAM 없이 한 장씩 넣은 비교(물체마다 한 장에 답 하나).

데이터 · 방법	찾은 비율 (%)	맞는 답 비율 (%)	장당 틀린 답	ADD-S AUC	BOP AR	처리 시간 (ms)
YCB-V · SAM-6D	90.2	90.9	0.41	90.0	78.7	1613
YCB-V · 제안, 텍스트	63.2	98.6	0.04	62.0	57.8	433
YCB-V · 제안, 정답 상자	80.9	100.0	0.00	79.0	74.5	528
YCB-M · SAM-6D	79.1	80.0	1.01	81.4	52.3	2197
YCB-M · 제안, 텍스트	42.6	94.0	0.14	41.9	29.7	395
YCB-M · 제안, 정답 상자	65.8	95.9	0.14	64.2	46.6	519
실내 · SAM-6D	52.4	43.0	3.39	53.6	33.3	2884
실내 · 제안, 텍스트	51.8	94.7	0.14	50.5	37.9	557
실내 · 제안, 정답 상자	52.2	97.6	0.06	49.4	37.2	497

AUC: 곡선 아래 면적(0~10 cm), AR: 평균 재현율. YCB-V 900장, YCB-M 925장, 실내 917장(의사 정답). 처리 시간은 한 장의 중앙값.

그림 5는 실내 주행에서 사람이 판정한 959개 (프레임, 물체) 칸으로 관문의 효과를 나눈 것이다. 색 분포 관문이 오검출을 가장 많이 줄였고(310 → 63), 외형 관문은 F1을 올렸으나 오검출이 96건으로 늘었으며, 배타 판정으로 28건이 되었다. 물체별 색조 보정을 더하면 F1은 0.841이다. 한 주행에서 자세 검증은 391건 중 288건을 채택하고 103건을 기각했는데, 기각된 자세는 모두 원래 구성의 1등 선택이었다면 그대로 출력되었을 것이다. 실제 조건에서 후보 생성부터 검증된 자세까지의 처리 시간은 1445 ms에서 377 ms로 줄었다.

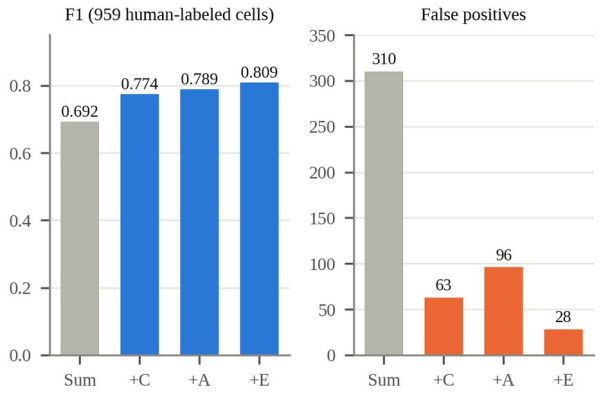


그림 5 관문을 하나씩 더했을 때의 효과(사람이 판정한 959칸). 가중 합(Sum)에 색 분포(+C), 외형(+A), 배타 판정(+E)을 차례로 더하였다.

6.2 | 촬영 시각 키프레임 고정

ORB-SLAM3 실시간 실행 기록 61회(실내 녹화 3종, 인식 결과 8964건, 촬영 후 도착까지 중앙값 1.45초·최대 6.6초)를 SLAM 자세, 키프레임 갱신, 인식 결과가 실제로 도착한 순서대로 다시 재생하면서 같은 결과를 네 가지 방식으로 등록하였다. (a) 도착 시각의 카메라 자세. (b) 촬영 시각으로 보간한 자세를 지도 좌표에 고정 저장(변환 보관 방식). (c) (b)와 같되

도착 시각의 키프레임에 대한 상대 자세로 저장. (d) 촬영 시각의 기준 키프레임에 대한 상대 자세로 저장(제안). 흩어짐은 모든 보정이 끝난 최종 지도에서 쟀다(표 2, 그림 6).

표 2 최종 지도에서 같은 물체 관측의 흩어짐.

등록 방식	중앙값 (cm)	상위 90% (cm)	10 cm 초과 (%)	상위 90%, 보정 (cm)	10 cm 초과, 보정 (%)
(a) 도착 시각 자세	9.67	31.94	48.2	294.0	63.8
(b) 촬영 시각, 지도 좌표 고정	3.08	9.70	9.2	299.8	30.8
(c) 촬영 시각, 도착 시각 키프레임	1.59	5.36	3.6	7.80	5.4
(d) 촬영 시각 키프레임 (제안)	**1.51**	**5.21**	**3.5**	**5.81**	**1.5**

보정: 결과를 기다리는 동안 2 cm를 넘는 지도 보정이 일어난 130건. 나머지 열은 전체 8964건.

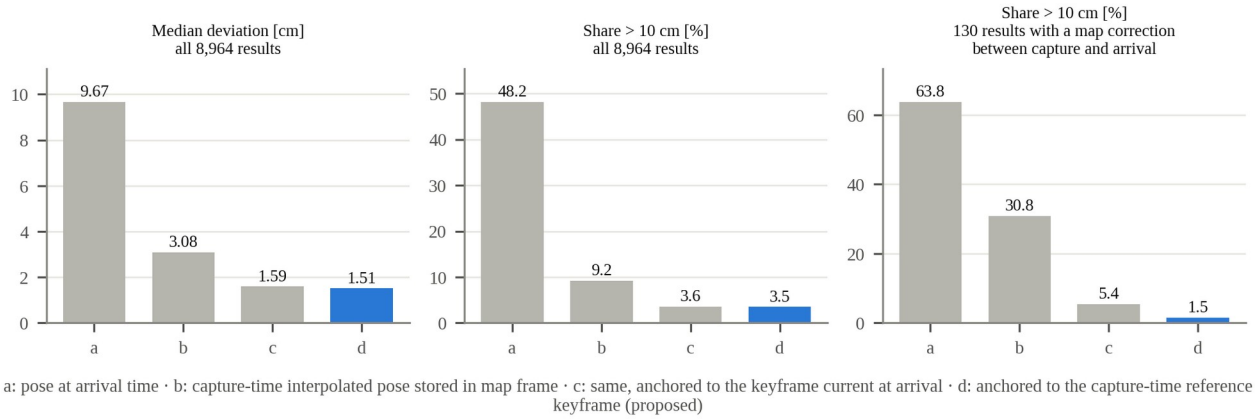


그림 6 실시간 실행 기록 61회에서 등록 방식 (a)~(d)의 비교.

촬영 시각을 쓰는 것만으로 오차의 대부분이 사라진다(a → b). 결과를 기다리는 동안 보정이 없으면 (c)와 (d)는 비슷하지만, 지도 보정이 끼어든 130건에서는 (d)만 일관성을 유지한다. 어두운 실내 8자 주행의 실행 18회에서 이 경우의 중앙값은 (c) 5.02 cm, (d) 0.58 cm였다. 같은 주행에서 루프 클로저 전후 같은 물체의 위치 차는 지도 좌표 저장 5.4 cm에서 0.8 cm로 줄었다. 91초 실내 주행에서는 키프레임 92개에 대해 최대 35 cm의 지도 보정이 96회 일어났으며, 고정 기능을 끄면 물체 세 개가 30~32 cm 떨어진 곳에 사본을 만들어 8개 물체가 11곳에 등록되었다. 대기 목록으로 지도에 올라가는 결과의 비율은 46%에서 100%가 되었다.

6.3 | 지속적 객체 기억

실내 주행 두 번에서 물체가 화면을 벗어나거나 가려졌다가 다시 보인 경우는 74건(화면 밖 39건)이었다. 이 가운데 72건이 지도의 같은 물체로 재연결되었고, 카메라 좌표로 연결하면 34건이었다(표 3). 실패 2건은 루프 클로저 전 누적 오차로 물체가 31~33 cm 어긋나 연결 범위를 넘은 경우이다. YCB-M(1초 이상 사라졌다 다시 보인 101건)에서 지도 기반 연결은 판정 대상의 91.8~97.9%에서 성공하여 카메라 좌표 연결보다 3~7%p 높았다.

표 3 실내 주행의 재연결 (키프레임 고정 객체 지도 / 지도 없이).

	밝은 실내 홀 8자, 151초	91초 실내
다시 보인 물체 (화면 밖)	35 (18)	39 (21)
같은 물체로 재연결	35 / 22	37 / 12
만든 물체 번호 (8종)	8 / 51	10 / 68
주행 뒤 같은 종류 중복	0 / 17	0 / 19

다수결로 방향을 정하면 20° 또는 50 mm를 넘게 어긋나 표시되는 비율이 평균 방식의 23.9%에서 9.3%로 줄었다. 중복 억제는 실제 물체에서 약 1.8 m 떨어진 오인식 사본을 관측 수 비 65:3, 72:3, 68:4에서 숨겼고 정상 물체의 표시는 바뀌지 않았다. 검출 가능 거리 조건을 쓰면 HDL-Graph-SLAM을 쓴 큰 8자 주행에서 작은 머그컵과 식혜캔의 표시 비율이 37%/46%에서 83%/82%로 올랐다. YCB-Video 실시간 재생에서 같은 SLAM과 지도를 쓸 때, SAM-6D 결과로는 12장면에서 중복 8건과 최대 85.1 mm의 위치 오차가 생겼으나 제안 인식기로는 중복이 없었고(정

답 상자 조건 최대 23.8 mm), 잘못 그린 상자는 프레임당 0.266개에서 0.001개로 줄었다.

6.4 | 인식 지연 보상 표시

그림 7은 마지막 결과를 카메라 좌표 그대로 유지하는 방식과 표시 시각의 SLAM 자세로 객체 지도를 그리는 방식을 비교한다. 모든 인식기와 데이터에서 화면 일치율이 올랐고, 인식기가 느릴수록 효과가 컸다. 한 장에 2.0초가 걸려 프레임의 8%만 처리하는 YCB-M의 SAM-6D에서 3.2%가 72.7%로 올랐다. 적응형 표시 지연을 쓰면 네트워크 지연이 큰 조건에서 보간할 SLAM 자세가 없는 표시 프레임의 비율이 100%에서 0%로 줄었다. 91초 실내 주행에서 인식은 초당 3.2장, 결과 도착 지연은 중앙값 0.70초였지만 화면은 15 Hz로 갱신되었고 SLAM은 입력된 1883프레임을 모두 추적하였다.

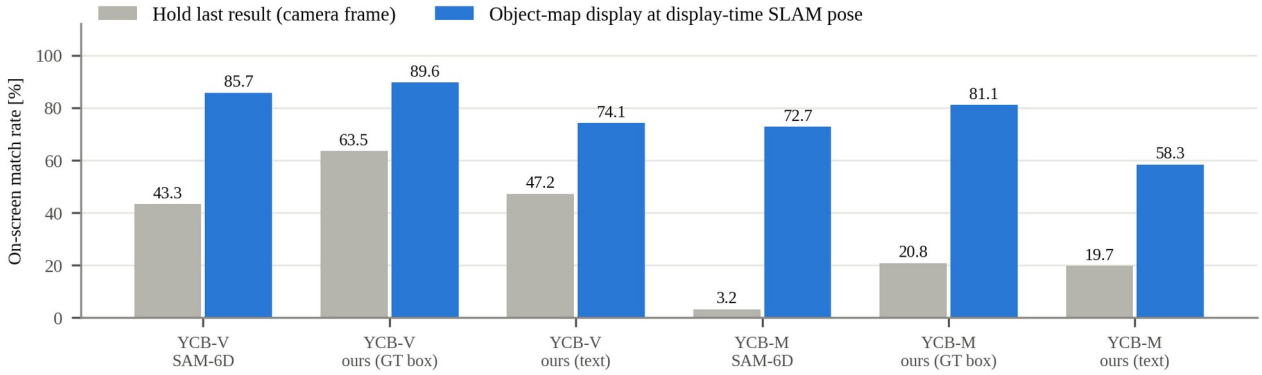


그림 7 YCB-Video(YCB-V)와 YCB-M에서의 화면 일치율.

6.5 | 한계

보이는 부분이 30~50%인 물체를 SAM-6D는 YCB-Video에서 46% 찾지만 제안 인식기는 3%만 찾는다. 상자 안에 가리는 물체가 함께 들어가 외형 점수가 떨어지고, 실루엣이 작아 겹침 기준을 넘지 못하기 때문이다. 잘 보이는 물체는 실내 주행에서 제안 75%, 원본 70%였다. 텍스트 프롬프트가 놓치는 물체가 있고, 무늬 없는 물체는 정답 상자를 주어도 관문을 넘지 못한다. 또한 키프레임 고정은 잘못된 보정도 그대로 따라간다. 큰 8자 주행을 같은 입력으로 40회 실행한 결과 2회는 루프 클로저가 정확한 지도를 망가뜨렸고(루프 보정 없는 LiDAR 스캔 정합 궤적 대비 흩어짐 중앙값 17.0 cm, 54.4 cm, 후자는 표시 물체가 8개에서 13개로 늘어남), 34회는 2.1~2.6 cm 안에서 일치하였다. 같은 종류 물체가 여러 개인 장면은 평가하지 않았다.

7 | 고찰

결과는 세 가지 효과를 나누어 보여 준다. 첫째는 *어느 시각*을 쓰는가이다. 도착 시각을 촬영 시각으로 바꾸는 것이 개선의 대부분을 차지한다(9.67 → 3.08 cm). 둘째는 *어느 좌표계*에 저장하는가이다. SLAM이 과거를 고치는 경우에는 키프레임 상대 저장이 필요하다. 셋째는 *어느 키프레임*인가이다. 도착 시각의 키프레임은 대부분 충분하지만, 결과를 기다리는 동안 보정이 일어나면 보정 전 카메라 자세와 보정 후 키프레임 자세를 섞게 되어 실패한다(10 cm 초과 30.8% → 5.4% → 1.5%). 제로샷 인식기는 느리고 지연이 들쭉날쭉하므로 이런 경우는 드물지 않으며, 본 실험에서 130번 일어났다. SLAM 장치는 이미 가진 정보를 내보내고 전송 시점만 정하므로, 이 방법은 ORB-SLAM3와 HDL-Graph-SLAM에서 보였듯 다른 키프레임·그래프 기반 SLAM과, 카메라 기준 자세를 내는 다른 인식기에도 적용할 수 있다.

8 | 결론

본 논문은 SLAM의 실시간성을 지키면서 늦게 도착하는 제로샷 6D 자세 추정 결과를 SLAM 지도에 올바르게 놓는 두 장치 시스템을 제안하였다. 키프레임 전송, 메타데이터 촬영 시각과 오프셋·외부 파라미터 동시 보정, 관문 검증 인식기, 존재확률 상태 관리, 인식 지연 보상 표시가 받쳐 주는 촬영 시각 키프레임 고정으로, 루프 클로저 뒤에도 일관된 객체 지도를 유지하고, 오래 사라졌던 물체를 다시 연결하며, 수초의 지연에도 화면 상자가 영상과 어긋나지 않았다. 앞으로 가림을 고려한 검증, 텍스트 검출기가 놓친 물체의 보충, 잘못된 루프 클로저의 검출, 같은 종류 물체가 여러 개인 장면을 다룰 계획이다.

참고문헌

1. C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, *ORB-SLAM3: an accurate open-source library for visual, visual–inertial, and multimap SLAM*, IEEE Trans. Robot. **37** (2021), no. 6, 1874–1890.
2. K. Koide, J. Miura, and E. Menegatti, *A portable three-dimensional LIDAR-based system for long-term and wide-area people behavior measurement*, Int. J. Adv. Robot. Syst. **16** (2019), no. 2.

3. J. Lin, L. Liu, D. Lu, and K. Jia, *SAM-6D: segment anything model meets zero-shot 6D object pose estimation*, (Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit., Seattle, WA, USA), 2024, pp. 27906–27916.
4. E. P. Örnek, Y. Labbé, B. Tekin, L. Ma, C. Keskin, C. Forster, and T. Hodaň, *FoundPose: unseen object pose estimation with foundation features*, (Proc. Eur. Conf. Comput. Vis., Milan, Italy), 2024.
5. V. N. Nguyen, T. Groueix, M. Salzmann, and V. Lepetit, *GigaPose: fast and robust novel object pose estimation via one correspondence*, (Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit., Seattle, WA, USA), 2024.
6. J. McCormac, R. Clark, M. Bloesch, A. J. Davison, and S. Leutenegger, *Fusion++: volumetric object-level SLAM*, (Proc. Int. Conf. 3D Vis., Verona, Italy), 2018.
7. T. Foote, *tf: the transform library*, (Proc. IEEE Int. Conf. Technol. Pract. Robot Appl., Woburn, MA, USA), 2013.
8. Y. Xiang, T. Schmidt, V. Narayanan, and D. Fox, *PoseCNN: a convolutional neural network for 6D object pose estimation in cluttered scenes*, (Proc. Robot. Sci. Syst., Pittsburgh, PA, USA), 2018.
9. T. Grenzdörffer, M. Günther, and J. Hertzberg, *YCB-M: a multi-camera RGB-D dataset for object recognition and 6DoF pose estimation*, (Proc. IEEE Int. Conf. Robot. Autom., Paris, France), 2020.
10. V. N. Nguyen, T. Groueix, G. Ponimatkin, V. Lepetit, and T. Hodaň, *CNOS: a strong baseline for CAD-based novel object segmentation*, (Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshops, Paris, France), 2023.
11. A. Kirillov et al., *Segment anything*, (Proc. IEEE/CVF Int. Conf. Comput. Vis., Paris, France), 2023.
12. X. Zhao et al., *Fast segment anything*, arXiv preprint, 2023. https://arxiv.org/abs/2306.12156
13. M. Oquab et al., *DINOv2: learning robust visual features without supervision*, Trans. Mach. Learn. Res. (2024).
14. B. Wen, W. Yang, J. Kautz, and S. Birchfield, *FoundationPose: unified 6D pose estimation and tracking of novel objects*, (Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit., Seattle, WA, USA), 2024.
15. T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, *YOLO-World: real-time open-vocabulary object detection*, (Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit., Seattle, WA, USA), 2024.
16. R. F. Salas-Moreno, R. A. Newcombe, H. Strasdat, P. H. J. Kelly, and A. J. Davison, *SLAM++: simultaneous localisation and mapping at the level of objects*, (Proc. IEEE Conf. Comput. Vis. Pattern Recognit., Portland, OR, USA), 2013.
17. S. Yang and S. Scherer, *CubeSLAM: monocular 3-D object SLAM*, IEEE Trans. Robot. **35** (2019), no. 4, 925–938.
18. L. Nicholson, M. Milford, and N. Sünderhauf, *QuadricSLAM: dual quadrics from object detections as landmarks in object-oriented SLAM*, IEEE Robot. Autom. Lett. **4** (2019), no. 1, 1–8.
19. Y. Labbé, J. Carpentier, M. Aubry, and J. Sivic, *CosyPose: consistent multi-view multi-object 6D pose estimation*, (Proc. Eur. Conf. Comput. Vis., Glasgow, UK), 2020.
20. J. Qian, V. Chatrath, J. Yang, J. Servos, A. P. Schoellig, and S. L. Waslander, *POCD: probabilistic object-level change detection and volumetric mapping in semi-static scenes*, (Proc. Robot. Sci. Syst., New York, NY, USA), 2022.
21. L. Schmid, M. Abate, Y. Chang, and L. Carlone, *Khronos: a unified approach for spatio-temporal metric-semantic SLAM in dynamic environments*, (Proc. Robot. Sci. Syst., Delft, Netherlands), 2024.
22. M. Rünz, M. Buffier, and L. Agapito, *MaskFusion: real-time recognition, tracking and reconstruction of multiple moving objects*, (Proc. IEEE Int. Symp. Mixed Augment. Real., Munich, Germany), 2018.
23. L. Liu, H. Li, and M. Gruteser, *Edge assisted real-time object detection for mobile augmented reality*, (Proc. Annu. Int. Conf. Mob. Comput. Netw., Los Cabos, Mexico), 2019.
24. C. Zhang et al., *Faster segment anything: towards lightweight SAM for mobile applications*, arXiv preprint, 2023. https://arxiv.org/abs/2306.14289
25. T. Hodaň et al., *BOP: benchmark for 6D object pose estimation*, (Proc. Eur. Conf. Comput. Vis., Munich, Germany), 2018.